

CHAPTER 1

Core Java & JVM Internals

Java Fundamentals · Collections Framework · Generics · Exceptions
Java 8–21 Features · Multithreading · JVM Architecture & GC

~20 Pages

40+ Q&A

4 Exercise Types

Interview Ready

Section 1 — Java Fundamentals

1.1 Primitives vs Wrapper Classes

Primitive	Bits	Range	Wrapper	Default
byte	8	-128 to 127	Byte	0
short	16	-32768 to 32767	Short	0
int	32	-2 ³¹ to 2 ³¹ -1	Integer	0
long	64	-2 ⁶³ to 2 ⁶³ -1	Long	0L
float	32	~3.4e38 (7 sig dig)	Float	0.0f
double	64	~1.8e308 (15 sig dig)	Double	0.0d
char	16	0-65535 Unicode	Character	\u0000
boolean	1	true/false	Boolean	false

- Integer cache: `Integer.valueOf(-128..127)` returns cached objects; outside range `==` is `FALSE` even for same value
- Autoboxing `NullPointerException`: `Integer x = null; int y = x;` — common interview trap
- Performance: prefer primitives in hot loops; wrapper collections (`List`) add boxing overhead

1.2 String, Immutability & equals/hashCode Contract

```
// String pool
String a = "hello"; // pool literal, interned
String b = "hello"; // same pool ref → a==b TRUE
String c = new String("hi"); // new heap object → c=="hi" FALSE
String d = c.intern(); // adds to pool → d=="hi" TRUE

// StringBuilder (single-thread) vs StringBuffer (synchronized, legacy)
StringBuilder sb = new StringBuilder();
for(int i=0;i<1000;i++) sb.append(i); // O(n) — CORRECT
// BAD: String s=""; for(...) s+=i; // O(n^2) — creates new object each time

// equals/hashCode contract
class Employee {
    int id; String name;
    @Override public boolean equals(Object o) {
        if(this==o) return true;
        if(!(o instanceof Employee e)) return false;
        return id==e.id && Objects.equals(name,e.name);
    }
    @Override public int hashCode() { return Objects.hash(id,name); }
}
```

- Contract: if `a.equals(b)` then `a.hashCode()==b.hashCode()` — violating this silently breaks `HashMap/HashSet`
- Five `equals()` properties: reflexive, symmetric, transitive, consistent, non-null

Section 2 — Collections Framework

2.1 Full Interface/Class Hierarchy

```
java.lang.Iterable<E>
+-- java.util.Collection<E>
+-- java.util.List<E> --> ArrayList, LinkedList, CopyOnWriteArrayList
+-- java.util.Set<E> --> HashSet, LinkedHashSet, TreeSet, EnumSet
+-- java.util.Queue<E>
+-- PriorityQueue
+-- java.util.Deque<E> --> ArrayDeque, LinkedList
java.util.Map<K,V> (does NOT extend Collection)
+-- HashMap, LinkedHashMap, TreeMap, EnumMap, WeakHashMap, IdentityHashMap
+-- java.util.concurrent.ConcurrentHashMap
```

2.2 LIST — ArrayList / LinkedList / CopyOnWriteArrayList

```
import java.util.ArrayList; import java.util.LinkedList;
import java.util.concurrent.CopyOnWriteArrayList;

// ArrayList: Object[] backed, O(1) get/set, amortized O(1) add, O(n) mid-insert
List<String> al = new ArrayList<>();
al.add("A"); al.add(0,"B"); al.set(0,"C"); al.get(0); al.remove(0);
al.contains("X"); al.indexOf("X"); al.subList(1,3);
al.sort(Comparator.naturalOrder()); al.removeIf(s->s.isEmpty());
List<String> imm = List.of("x","y","z"); // unmodifiable factory (Java 9+)

// LinkedList: doubly-linked + implements Deque; O(1) ends, O(n) index
LinkedList<String> ll = new LinkedList<>();
ll.addFirst("h"); ll.addLast("t"); ll.removeFirst(); ll.peekLast();

// CopyOnWriteArrayList: copies array on write; iterator never throws CME
CopyOnWriteArrayList<String> cow = new CopyOnWriteArrayList<>();
cow.add("safe"); // good for read-heavy, infrequently modified lists
```

2.3 SET — HashSet / LinkedHashSet / TreeSet / EnumSet

```
import java.util.*;

HashSet<String> hs = new HashSet<>(); // O(1), no order
LinkedHashSet<String> lhs = new LinkedHashSet<>(); // insertion-order
TreeSet<Integer> ts = new TreeSet<>(); // sorted, O(log n)

// NavigableSet on TreeSet
ts.addAll(Arrays.asList(5,2,8,1,9,3));
ts.floor(6)/5*/; ts.ceiling(6)/8*/; ts.headSet(5)/[1,2,3]*/; ts.pollFirst()/*1*/;

// Set algebra
Set<Integer> a = new HashSet<>(Set.of(1,2,3,4));
Set<Integer> b = new HashSet<>(Set.of(3,4,5,6));
new HashSet<>(a).addAll(b); // union {1,2,3,4,5,6}
new HashSet<>(a).retainAll(b); // intersection {3,4}
new HashSet<>(a).removeAll(b); // difference {1,2}

// EnumSet – bit-vector, fastest set for enums
enum Day{MON,TUE,WED,THU,FRI,SAT,SUN}
EnumSet<Day> weekdays = EnumSet.range(Day.MON,Day.FRI);
EnumSet<Day> weekend = EnumSet.of(Day.SAT,Day.SUN);
```

2.4 MAP — HashMap / LinkedHashMap / TreeMap / ConcurrentHashMap

```
import java.util.*; import java.util.concurrent.ConcurrentHashMap;

// HashMap: bucket array, O(1) avg; treeify bin >= 8 (Java 8+); load factor 0.75
```

```

Map<String,Integer> hm = new HashMap<>();
hm.put("a",1); hm.get("a"); hm.getDefault("b",0);
hm.putIfAbsent("c",3); hm.computeIfAbsent("d",k->k.length());
hm.merge("a",10,Integer::sum); // a -> 11
hm.forEach((k,v)->System.out.println(k+"="+v));
for(Map.Entry<String,Integer> e : hm.entrySet()) { /**/ }
// LinkedHashMap LRU cache (accessOrder=true)
Map<String,String> lru = new LinkedHashMap<>(16,0.75f,true) {
protected boolean removeEldestEntry(Map.Entry<String,String> e){
return size()>100; // evict LRU when > 100 entries
}
};
// TreeMap: sorted + NavigableMap
TreeMap<String,Integer> tm = new TreeMap<>();
tm.floorKey("m"); tm.ceilingKey("m"); tm.headMap("m"); tm.pollFirstEntry();
// ConcurrentHashMap: per-bin CAS+sync, no full-table lock, null key NOT allowed
ConcurrentHashMap<String,Integer> chm = new ConcurrentHashMap<>();
chm.put("x",1); chm.computeIfAbsent("y",k->42); // atomic per key

```

2.5 QUEUE/DEQUE — PriorityQueue / ArrayDeque / BlockingQueue

```

import java.util.*; import java.util.concurrent.*;
// PriorityQueue: min-heap; O(log n) offer/poll; O(1) peek
PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap
PriorityQueue<Integer> max = new PriorityQueue<>(Comparator.reverseOrder()); // max
pq.offer(3); pq.offer(1); pq.poll();/*1*/ pq.peek();/*min*/
// ArrayDeque: preferred Stack AND Queue (faster than Stack/LinkedList)
ArrayDeque<String> dq = new ArrayDeque<>();
dq.push("a"); dq.pop(); // LIFO stack
dq.offer("b"); dq.poll(); // FIFO queue
dq.offerFirst("c"); dq.peekLast();
// BlockingQueue: producer/consumer pattern
BlockingQueue<String> bq = new LinkedBlockingQueue<>(100);
// Producer: bq.put("task"); // blocks if full
// Consumer: String t = bq.take(); // blocks if empty
// Non-block: bq.offer("t",1,SECONDS); bq.poll(1,SECONDS);

```

2.6 Fail-fast vs Fail-safe & Collections Utility

```

// Fail-fast: modCount check -> throws ConcurrentModificationException
List<String> list = new ArrayList<>(Arrays.asList("a","b","c"));
// BAD: for(String s:list){ if(s.equals("b")) list.remove(s); } // CME!
list.removeIf(s->s.equals("b")); // SAFE
// Fail-safe: CopyOnWriteArrayList, ConcurrentHashMap iterate over snapshot
// Collections utility class
Collections.sort(list); Collections.sort(list,Comparator.reverseOrder());
Collections.binarySearch(list,"a"); // list must be sorted first
Collections.reverse(list); Collections.shuffle(list);
List<String> ro = Collections.unmodifiableList(list); // read-only view
List<String> sync = Collections.synchronizedList(list); // legacy sync wrap
Collections.frequency(list,"a"); Collections.disjoint(list,other);
Collections.min(list); Collections.max(list);

```

2.7 Choosing the Right Collection — Decision Table

Use Case	Best Choice	Why
Index access / iteration	ArrayList	O(1) get; cache friendly

LIFO Stack	ArrayDeque (push/pop)	No sync overhead vs Stack
FIFO Queue	ArrayDeque (offer/poll)	Faster than LinkedList
Unique, no order	HashSet	O(1) add/contains
Unique, insertion order	LinkedHashSet	O(1) + ordered
Unique, sorted + range	TreeSet	NavigableSet O(log n)
Key-value fast lookup	HashMap	O(1) avg
Key-value sorted	TreeMap	NavigableMap O(log n)
LRU Cache	LinkedHashMap	accessOrder + removeEldest
Thread-safe map	ConcurrentHashMap	No full-table lock
Priority processing	PriorityQueue	Min/max heap O(log n)
Producer-consumer	LinkedBlockingQueue	Blocking put/take, bounded
Read-mostly thread-safe list	CopyOnWriteArrayList	Snapshot iteration
Enum keys map	EnumMap	Array-backed O(1), zero GC

Section 3 — Generics

```
// Generic class & bounded type parameter
class Pair<A,B> { A first; B second; }
public static <T extends Comparable<T>> T max(T a, T b){
return a.compareTo(b)>=0 ? a : b;
}

// Type erasure: compile-time only; erased to Object (or bound) at runtime
// List<String>.class is INVALID. Cannot: new T(); instanceof List<String>; T[].
// Wildcards – PECS (Producer Extends, Consumer Super)
void readFrom(List<? extends Number> src) { Number n = src.get(0); } // read OK, no write
void writeTo(List<? super Integer> dst) { dst.add(42); } // write OK, no typed read
// Unbounded: void printAll(List<?> list) { list.forEach(System.out::println); }
// Generic method with multiple bounds
public static <T extends Comparable<T> & Serializable> void process(T val){ /**/ }
// Bridge method (compiler-generated for erasure + polymorphism)
// class StringList extends ArrayList<String> { void add(String s){...} }
// Compiler adds: void add(Object o){ add((String)o); } -- bridge
```

- PECS mnemonic: if the list PRODUCES values you READ → extends; if it CONSUMES values you WRITE → super
- Cannot use primitives as type params (use Integer, not int); autoboxing bridges the gap
- Array of generic type: use (T[]) new Object[n] or Array.newInstance(clazz, n)

Section 4 — Exception Handling

```
// Throwable -> Error (OOM, StackOverflow -- do not catch)
// -> Exception -> RuntimeException (unchecked: NullPointerException, ArrayIndex...)
// -> IOException, SQLException... (checked -- must declare/catch)
// try-with-resources (Java 7+) -- auto-closes AutoCloseable in reverse order
try(Connection c=ds.getConnection(); PreparedStatement ps=c.prepareStatement(sql)){
ps.executeQuery();
} // ps closed first, then c; suppressed exceptions if close() also throws
// Multi-catch (Java 7+)
catch(IOException | SQLException e){ log.error("Data error",e); }
// Custom exception
class BookingConflictException extends RuntimeException {
private final String bookingId;
public BookingConflictException(String id,String msg){
super(msg); this.bookingId=id;
}
}
// Throw: throw new BookingConflictException(id,"Already booked by another user");
// Anti-patterns (AVOID)
catch(Exception e){} // swallowing -- hides bugs
catch(Exception e){ throw e; } // useless re-throw
if(x==null) throw new Exception("null"); // use specific type
try{Integer.parseInt(s);}catch(Exception e){} // flow control via exception
```

- Checked: recoverable conditions (file not found, DB connection failed) — forces caller to handle
- Unchecked: programming errors (NullPointerException, IndexOutOfBoundsException) — document, don't always catch
- finally always runs (even after return); avoid return inside finally — it suppresses the original return

Section 5 — Java 8–21 Key Features

```
// Functional Interfaces & Lambdas
Predicate<String> notEmpty = s -> !s.isEmpty();
Function<String,Integer> len = String::length; // method reference
Consumer<String> print = System.out::println;
Supplier<List<String>> factory = ArrayList::new;
BinaryOperator<Integer> add = Integer::sum;

// Stream API -- lazy: intermediate ops build pipeline; terminal triggers execution
List<String> names = List.of("Alice","Bob","Charlie","Anna");
List<String> result = names.stream()
    .filter(n->n.startsWith("A")) // intermediate (lazy)
    .map(String::toUpperCase) // intermediate (lazy)
    .sorted() // intermediate (lazy)
    .collect(Collectors.toList()); // TERMINAL -- triggers execution

Map<Integer,List<String>> byLen = names.stream().collect(Collectors.groupingBy(String::length));
String joined = names.stream().collect(Collectors.joining(", ","[","]"));
List<Integer> flat =
    Stream.of(List.of(1,2),List.of(3,4)).flatMap(Collection::stream).collect(Collectors.toList());

// Optional -- avoid NullPointerException
Optional<String> opt = Optional.ofNullable(getValue());
String v = opt.map(String::toUpperCase).orElse("default");
// orElse: eager (always evaluates arg); orElseGet: lazy (supplier called only if empty)
opt.orElseGet(()->expensiveDefault()); // preferred
opt.orElseThrow(()->new NotFoundException("id"));

// Records (Java 16)
record Point(int x,int y){} // immutable; auto: constructor/getters>equals/hashCode/toString
record Range(int s,int e){ Range{ if(s>e) throw new IllegalArgumentException(); } }

// Sealed classes (Java 17) + pattern matching switch (Java 21)
sealed interface Shape permits Circle,Rectangle{}
record Circle(double r) implements Shape{}
record Rectangle(double w,double h) implements Shape{}
double area = switch(shape){
    case Circle c -> Math.PI*c.r()*c.r();
    case Rectangle r -> r.w()*r.h();
};

// Virtual Threads (Java 21) -- JVM-managed, millions possible
try(var pool=Executors.newVirtualThreadPerTaskExecutor()){
    IntStream.range(0,10_000).forEach(i->pool.submit(()->processRequest(i)));
}
```

- Streams are lazy — filter/map don't execute until a terminal op is called; enables short-circuit optimisation
- Virtual threads park cheaply on blocking IO without consuming an OS thread — ideal for IO-bound microservices
- Parallel streams use ForkJoinPool.commonPool() — great for CPU-bound, stateless ops; avoid for IO or stateful code

Section 6 — Multithreading & Concurrency

```
// Thread states: NEW -> RUNNABLE -> BLOCKED/WAITING/TIMED_WAITING -> TERMINATED
// synchronized (intrinsic monitor)
class Counter {
    private int count=0;
    public synchronized void increment(){ count++; }
    public synchronized int get(){ return count; }
}

// volatile -- visibility (NOT atomicity)
volatile boolean running=true; // all threads see latest write immediately
```

```
// count++ is still NOT atomic on volatile int
AtomicInteger ai = new AtomicInteger(0); ai.incrementAndGet(); // atomic
// ReentrantLock -- explicit, fair, interruptible, tryLock
ReentrantLock lock = new ReentrantLock(true); // fair
lock.lock();
try{ /* critical section */ } finally{ lock.unlock(); } // ALWAYS in finally
// ReadWriteLock: many concurrent readers, exclusive writer
ReadWriteLock rw = new ReentrantReadWriteLock();
rw.readLock().lock(); try{ /* read */ } finally{rw.readLock().unlock();}
rw.writeLock().lock(); try{ /* write */ } finally{rw.writeLock().unlock();}
// CompletableFuture (non-blocking, composable)
CompletableFuture<String> cf = CompletableFuture
    .supplyAsync(()->fetchUser(id)) // async
    .thenApply(u->u.getName()) // transform
    .thenCompose(n->fetchOrders(n)) // chain another async
    .exceptionally(ex->"default"); // fallback
CompletableFuture.allOf(cf1,cf2,cf3).join();
// ThreadLocal -- thread-isolated storage; MUST remove() in thread pools
ThreadLocal<SimpleDateFormat> tl = ThreadLocal.withInitial(SimpleDateFormat::new);
try{ tl.set(sdf); doWork(); } finally{ tl.remove(); } // prevent memory leak
// CAS & ABA problem
AtomicInteger cas = new AtomicInteger(0);
cas.compareAndSet(0,1); // atomic if current==0, set to 1
// ABA: 0->1->0; CAS sees 0 again and succeeds -- use AtomicStampedReference<V>
// Thread pool sizing
// CPU-bound: N_cpu + 1
// IO-bound: N_cpu * (1 + wait/compute)
```


Section 7 — JVM Internals

7.1 Class Loading

```
// Parent-delegation: child asks parent first
// Bootstrap CL (native, loads java.base)
// +--> Platform CL (java.se modules)
// +--> Application CL (classpath)
// +--> Custom CL (plugins, hot-reload)
// Phases: Loading -> Linking(Verify->Prepare->Resolve) -> Initialization
// Class.forName("X") -- loads + initializes
// classLoader.loadClass("X") -- loads only (no static init)

class HotReloadCL extends ClassLoader {
protected Class<?> findClass(String name) throws ClassNotFoundException {
byte[] bytes = readClassBytes(name); // from disk/network
return defineClass(name,bytes,0,bytes.length);
}
}
```

7.2 JIT Compilation — Tiered

Level	Compiler	Purpose
0	Interpreter	Pure bytecode interpretation; baseline
1-2	C1 (client)	Fast compile; simple opts; collects profiling data
3	C1 + full profile	Call-frequency, branch data — feeds C2
4	C2 (server)	Aggressive: method inlining, escape analysis, loop unrolling

- Escape analysis: if object doesn't escape method/thread → stack allocation (no GC), scalar replacement, lock elision

7.3 Garbage Collection

GC	Default Since	Pauses	Best For	Key Flag
Serial	JDK 1	Long STW	Single-core, tiny heaps	-XX:+UseSerialGC
Parallel	JDK 5	Medium STW	Throughput (batch jobs)	-XX:+UseParallelGC
G1GC	JDK 9	~100-200ms	Balanced; large heaps	-XX:+UseG1GC
ZGC	JDK 15	<1ms	Ultra-low latency (TB heaps)	-XX:+UseZGC
Shenandoah	JDK 12	<10ms	Low-latency, Red Hat JDK	-XX:+UseShenandoahGC

```
// Heap layout
// Young (Eden + S0 + S1) --> Old Gen --> Metaspace (native memory, off-heap)
// Minor GC: clears Eden+survivor; promotes surviving objects to OldGen
// Major/Full GC: collects whole heap; stop-the-world (longer pause)

// Key JVM flags
-Xms512m -Xmx4g // initial/max heap
-XX:+UseG1GC -XX:MaxGCPauseMillis=200 // G1 with 200ms target
-XX:+UseZGC // sub-ms pauses
-Xlog:gc*:file=gc.log // GC logging (Java 9+)
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/heap.hprof

// Memory leak diagnosis
// 1. Monitor heap via JMX / Micrometer (heap used grows unbounded)
// 2. jmap -dump:format=b,file=heap.hprof <pid> -- take heap dump
// 3. Eclipse MAT Leak Suspects -- find largest retained object graphs
// Common leaks: static Maps, ThreadLocal not removed, listener not unregistered
```

JVM Memory Area	Contents	Error
-----------------	----------	-------

Heap	Objects, arrays	OutOfMemoryError: Java heap space
Metaspace (native)	Class metadata, interned strings	OutOfMemoryError: Metaspace
Thread Stack	Frames, local vars	StackOverflowError (deep recursion)
Code Cache	JIT compiled native code	CodeCache is full (rare)
Direct Buffer	NIO ByteBuffers	OutOfMemoryError: Direct buffer memory

Interview Q&A — 40 Questions

Java Fundamentals

Q1. What is the difference between `==` and `equals()` for Strings?

`==` compares object references. String literals from the pool may share the same reference so `==` returns true, but new `String("hello") == "hello"` is false — different heap objects. Always use `equals()` for content comparison, `equalsIgnoreCase()` for case-insensitive.

Q2. Why must you override `hashCode()` when you override `equals()`?

Contract: if `a.equals(b)` then `a.hashCode()==b.hashCode()`. `HashMap/HashSet` use `hashCode()` to find the bucket, then `equals()` to confirm the key. Override only `equals()` without `hashCode()`, and equal objects land in different buckets — `HashMap.get()` returns null for a key that logically exists.

Q3. What are the autoboxing pitfalls?

1) `NullPointerException`: `Integer x=null; int y=x;` 2) Reference equality trap: `Integer a=200; Integer b=200; a==b` is FALSE (outside -128..127 cache). 3) Performance: boxing in tight loops creates many short-lived objects, adding GC pressure. 4) Overloaded method resolution can be ambiguous.

Q4. Is Java pass-by-value or pass-by-reference?

Always pass-by-value. For primitives, the value is copied. For objects, the reference (memory address) is copied — the caller's reference is unaffected by reassigning the parameter, but you can mutate the object's state through the copied reference.

Q5. Why is String immutable?

Security (class names, DB credentials can't be tampered); String pool efficiency (multiple references to one literal); thread-safety without synchronisation; `hashCode` caching (computed once, reused in `HashMap` lookups).

Collections

Q6. How does HashMap work internally?

Maintains `Node[]` bucket array. On `put(k,v)`: `hash=hashCode(k)^(h>>>16)`; `index=hash & (cap-1)`. Empty bucket → new Node; collision → linked list. Java 8+: when bin size ≥ 8 , list converts to red-black tree ($O(\log n)$). Resize when `size > cap*0.75`: doubles capacity, rehashes all entries.

Q7. Why is HashMap not thread-safe?

Concurrent puts cause data loss (two threads write the same bucket). Java 7: concurrent resize could create a circular linked list causing infinite loops in `get()`. Java 8+: data races still cause silent corruption. Use `ConcurrentHashMap` — per-bin CAS + synchronisation without locking the whole table.

Q8. Explain fail-fast vs fail-safe iterators.

Fail-fast (`ArrayList`, `HashMap`): tracks `modCount`; throws `ConcurrentModificationException` if collection is modified during iteration — detects bugs early. Fail-safe (`CopyOnWriteArrayList`, `ConcurrentHashMap`): iterates a snapshot; never throws CME but may not see the latest writes.

Q9. When would you use LinkedList over ArrayList?

Rarely — `ArrayDeque` beats `LinkedList` for Deque operations. `LinkedList` wins only when you need frequent $O(1)$ insertions/deletions at both ends AND you're also using `ListIterator` to insert mid-list. For most use cases, `ArrayList`'s cache locality and lower memory overhead win.

Q10. How does the LRU cache pattern work with LinkedHashMap?

Construct with `accessOrder=true` (3rd constructor arg). Every `get()` moves the accessed entry to the tail. Override `removeEldestEntry()` to return `size()>MAX_SIZE` — when true, the head entry (least recently used) is automatically evicted. Gives $O(1)$ `get/put` with automatic LRU eviction.

Q11. Difference between poll() and remove() in Queue?

poll() returns null if the queue is empty (safe). remove() throws NoSuchElementException if empty. Same pattern: peek() vs element() (peek=null on empty, element=exception). Always prefer poll()/peek() in production to avoid unchecked exceptions.

Q12. How does ConcurrentHashMap achieve thread safety without full-table lock?

Java 8+: uses CAS for empty buckets (lock-free insert) and synchronized on the first node of each bin for collisions. Reads are entirely lock-free. Compute/merge/computeIfAbsent operations are atomic per key. This allows many concurrent readers and writers at different keys without contention.

Generics**Q13. What is type erasure and why does Java use it?**

Generic type parameters are enforced at compile time then erased to their bounds (usually Object) at runtime. List and List are both just List at runtime. Done for backward compatibility with pre-Java-5 raw-type code. Consequences: cannot do instanceof List, new T(), overload by generic type alone.

Q14. Explain PECS with an example.

Producer Extends, Consumer Super. List — you can only READ from it (producer of Number values), not write (compiler doesn't know exact type). List — you can only WRITE Integer (consumer), not read as a typed value. Use concrete type List if you need both.

Java 8–21 / Streams**Q15. How does Stream lazy evaluation work?**

Intermediate ops (filter, map, flatMap, sorted, limit) don't execute; they build a lazy pipeline. Execution begins only when a terminal op (collect, forEach, count, reduce, findFirst, anyMatch) is called. Short-circuit: findFirst stops after the first match; anyMatch stops at first true.

Q16. Difference between map() and flatMap()?

map() is 1-to-1: each element produces exactly one output element. flatMap() is 1-to-many: each element produces a Stream, and all streams are merged (flattened) into one. Example: stream of sentences.flatMap(s -> Arrays.stream(s.split(" "))) produces a stream of words.

Q17. Difference between orElse() and orElseGet()?

orElse(value) always evaluates the default argument even when Optional has a value — costly if default involves method calls. orElseGet(supplier) evaluates the supplier lazily only when Optional is empty. Rule: prefer orElseGet for expensive defaults; orElse only for cheap constants.

Q18. What are virtual threads and when should you use them?

Java 21 virtual threads are JVM-managed (~1 KB stack), unlike platform threads (~1 MB OS thread). They park cheaply on blocking IO without tying up an OS thread, allowing millions of concurrent tasks. Use Executors.newVirtualThreadPerTaskExecutor() for IO-bound services. Not beneficial for CPU-bound computation.

Q19. What are records and when are they appropriate?

Records (Java 16) are immutable data carriers — auto-generate canonical constructor, accessor methods (fieldname(), not getX()), equals/hashCode/toString. Implicitly final. Use for DTOs, value objects, API response types. Not for entities needing mutation or ORM mapping (no default constructor, no setters).

Q20. What are sealed classes used for?

Restrict which classes can implement/extend a type. The compiler knows the complete set of subtypes, enabling exhaustive pattern matching in switch expressions — if you miss a case, compilation fails. Ideal for algebraic data types: Result = Success | Failure, Shape = Circle | Rectangle | Triangle.

Multithreading & Concurrency

Q21. What does volatile guarantee?

Visibility: writes to volatile are immediately visible to all threads (bypasses CPU cache/registers). Happens-before: volatile write happens-before any subsequent read of the same variable. Does NOT guarantee atomicity — `volatile int count++` is still read-increment-write, three non-atomic steps. Use `AtomicInteger` for atomic increment.

Q22. What is the ABA problem?

CAS checks `current==expected` then swaps. ABA: thread reads A; another changes A->B->A; first thread CAS succeeds — but the underlying state has changed. Fix: `AtomicStampedReference` tracks a version stamp alongside the value, so `A-stamp1 != A-stamp2`, and CAS fails correctly.

Q23. How does `CompletableFuture` differ from `Future`?

`Future` requires blocking `.get()` to retrieve a result. `CompletableFuture` is non-blocking and composable: `thenApply` (transform), `thenCompose` (flat-map another async), `thenCombine` (zip two futures), exceptionally (fallback), `allOf/anyOf` (coordinate many). Enables reactive-style pipelines without blocking threads.

Q24. Explain thread pool sizing for CPU-bound vs IO-bound.

CPU-bound: `threads = N_cpu + 1` (one extra for preemption). More threads just add context-switching overhead.
IO-bound: `threads = N_cpu * (1 + wait/compute)`. A thread spending 90% waiting can coexist with 10x more threads than CPUs. Java 21 virtual threads make IO-bound sizing trivial — one virtual thread per task.

Q25. When would you use `ReadWriteLock` vs `StampedLock`?

`ReadWriteLock`: clear semantics, allows many concurrent readers or one exclusive writer. `StampedLock` (Java 8): adds optimistic read — `tryOptimisticRead()` returns a stamp; after reading, `validate()` checks if a write occurred; if invalid, fall back to full read lock. Faster than `ReadWriteLock` when reads dominate and writes are rare.

JVM Internals**Q26. Explain the class loading delegation model.**

Child class loader asks its parent first before trying itself: Bootstrap (native, java.base) -> Platform (java.se) -> Application (classpath). This prevents user code from overriding core Java classes. OSGi and application servers override this with custom class loaders for module isolation and hot-reloading.

Q27. How does G1GC work?

Divides heap into equal regions (1-32 MB each), dynamically assigned as Eden/Survivor/Old. Concurrent marking identifies live objects while application runs. Mixed GC collects Young + selected Old regions to meet the pause target (`-XX:MaxGCPauseMillis`). Humongous regions handle large objects. Default GC since Java 9.

Q28. What JVM flags would you use for ultra-low latency?

`-XX:+UseZGC` (Java 15+): concurrent compaction, sub-millisecond pauses, colored pointers. Or `-XX:+UseShenandoahGC`. For G1: `-XX:MaxGCPauseMillis=50 -XX:G1HeapRegionSize=16m`. Also `-XX:+AlwaysPreTouch` (pre-warm pages), `-XX:+UseLargePages`. Avoid Full GC: pre-size heap `-Xms=-Xmx`.

Q29. How do you diagnose a Java memory leak in production?

1) Monitor heap trend with JMX/Micrometer (heap used grows over time without levelling off). 2) Take heap dump: `jmap -dump:format=b,file=heap.hprof` or `-XX:+HeapDumpOnOutOfMemoryError`. 3) Analyse with Eclipse MAT: Leak Suspects report shows largest retained sets. Common culprits: unbounded static collections, `ThreadLocal` not removed, event listeners not unregistered.

Q30. What is Metaspace and why was PermGen removed?

PermGen (pre-Java 8) stored class metadata in the heap with a fixed max size — `OutOfMemoryError: PermGen space` was common in class-heavy apps. Metaspace (Java 8+) uses native (OS) memory, grows dynamically by default (cap with `-XX:MaxMetaspaceSize`). Eliminates PermGen OOM and allows more flexible class loading at scale.

Quick-Fire (31-40)

Q31. Difference between Comparator and Comparable?

Comparable: class implements it for natural ordering (compareTo). Comparator: external comparison strategy passed to sort/TreeSet — allows multiple orderings without modifying the class.

Comparator.comparing(Person::getAge).thenComparing(Person::getName) enables fluent multi-level sort.

Q32. What is the difference between Iterator.remove() and Collection.remove()?

Iterator.remove() removes the last element returned by next() and safely updates modCount — the only safe way to remove during iteration. Collection.remove() during a for-each loop triggers a ConcurrentModificationException with fail-fast iterators. Prefer list.removeIf(predicate) for bulk removal.

Q33. When does HashMap resize and at what cost?

Resizes when size > capacity * loadFactor (default $16 * 0.75 = 12$ entries). Doubles capacity and rehashes all entries — $O(n)$ cost. Pre-size to avoid: new HashMap<>(expectedSize / 0.75 + 1). For 1000 entries: new HashMap<>(1334) avoids any resize.

Q34. Explain EnumMap and when to use it.

Backed by an array indexed by enum ordinal — $O(1)$ all ops, no hashing, extremely memory-efficient. Always prefer EnumMap over HashMap. EnumSet is similarly optimal for enum sets.

Q35. What is a ThreadLocal memory leak?

Thread pool threads are reused. If a task sets ThreadLocal but doesn't call remove(), the value persists on the thread and is never GC'd (ThreadLocalMap holds a strong reference). Pattern: try { tl.set(value); doWork(); } finally { tl.remove(); } — always in a finally block.

Q36. Difference between Collections.synchronizedList() and CopyOnWriteArrayList?

synchronizedList wraps every method in synchronized but still throws CME if you iterate without external synchronization. CopyOnWriteArrayList copies the array on every write — iterators always see a consistent snapshot, never throw CME. Use COW only for read-heavily lists that are rarely modified (writes are expensive $O(n)$ copy).

Q37. What does CountdownLatch vs CyclicBarrier do?

CountDownLatch: one-time gate. Threads await() until count reaches 0 via countdown() calls. Cannot be reset.

CyclicBarrier: reusable. N threads all call await() and block until all N arrive; then all proceed together. Good for iterative parallel phases. Phaser is the most flexible, supporting dynamic party count.

Q38. Explain escape analysis.

JIT analyses whether an object's reference escapes the creating method or thread. If not: stack allocation instead of heap (no GC pressure), scalar replacement (decompose into primitive fields), lock elision (synchronized on non-escaped object is removed). Enabled by default; use -XX:+DoEscapeAnalysis (on by default with C2).

Q39. What are the Java Memory Model happens-before rules?

Key rules: 1) Program order (single thread). 2) Monitor unlock -> subsequent lock of same monitor. 3) Volatile write -> subsequent read of same variable. 4) Thread.start() -> actions in started thread. 5) Thread actions -> Thread.join() return. These guarantee cross-thread visibility without additional synchronization.

Q40. Difference between Executor, ExecutorService, ScheduledExecutorService?

Executor: execute(Runnable) only. ExecutorService: adds submit() returning Future, shutdown(), awaitTermination(), invokeAll/Any(). ScheduledExecutorService: adds schedule() for one-shot delayed tasks, scheduleAtFixedRate() and scheduleWithFixedDelay() for periodic tasks.

Exercises

Part A — Coding Challenges

- C1.** Implement a thread-safe generic LRU cache `LRUCache` using `LinkedHashMap` with configurable max capacity. Include `get(key)` returning null on miss, and `put(key,value)` with automatic LRU eviction.
- C2.** Write a Stream pipeline on `List` (fields: name, dept, salary) that: (a) groups by department, (b) finds the highest-paid employee per department. Return `Map`.
- C3.** Implement a generic bounded blocking queue `MyBlockingQueue` from scratch using `ReentrantLock` + two `Condition` objects (`notFull`, `notEmpty`). Support `offer(T,timeout,unit)` and `poll(timeout,unit)`.
- C4.** Find all duplicate elements in a `List` using Streams (no loops). Return `Set` of duplicates.
- C5.** Implement a retry utility `Retry.execute(Supplier, maxAttempts, backoffMillis)` using generics and lambdas. Retry on any exception with exponential backoff; throw on exhaustion.

Part B — Debug & Fix

D1 – Broken equals/hashCode

```
class Order{int id;String ref;}
Map<Order,String> map = new HashMap<>();
map.put(new Order(1,"X"),"val");
System.out.println(map.get(new Order(1,"X"))); // prints null -- WHY?
```

Fix: `Order` doesn't override `equals`/`hashCode`. Two `Order(1,"X")` objects are different identity objects. `HashMap` uses `Object.hashCode()` → different buckets. Fix: `@Override equals()` using `id+ref` and `hashCode()` using `Objects.hash(id,ref)`.

D2 – ConcurrentModificationException

```
for(String s : list){
if(s.contains("bad")) list.remove(s); // THROWS CME
}
```

Fix: `for-each` uses `ArrayList`'s fail-fast iterator; `remove()` increments `modCount`, `next()` detects mismatch. Fix: `list.removeIf(s -> s.contains("bad"))`; — safe single-pass removal.

D3 – ThreadLocal Memory Leak

```
ExecutorService pool = Executors.newFixedThreadPool(10);
pool.submit(() -> {
threadLocal.set(bigObject); // NEVER removed
processRequest();
});
```

Fix: Thread pool reuses threads; `bigObject` is never GC'd. Fix: `try { threadLocal.set(bigObject); processRequest(); } finally { threadLocal.remove(); }`

D4 – Optional misuse

```
Optional<User> opt = findUser(id);
String name = opt.get().getName(); // NoSuchElementException if empty!
```

Fix: Never call `get()` without checking `isPresent()`. Fix: `String name = opt.map(User::getName).orElseThrow() -> new UserNotFoundException(id);`

Part C — Design Problems

- DS1.** Design a type-safe in-memory `EventBus`: `subscribe(Class eventType, Consumer handler)` and `publish(T event)`. Thread-safe. Use `ConcurrentHashMap`, `CopyOnWriteArrayList`>>.
- DS2.** Design a generic `Cache` with TTL eviction. Use `ConcurrentHashMap` internally. Add a background `ScheduledExecutorService` that scans and removes expired entries every 60 seconds.

DS3. Design a Pipeline builder using generics and Function composition. Support: `Pipeline.of(Function).then(Function).then(Function).execute(T input)`.

Part D — MCQ / Scenario Quiz

M1. What does `Collections.unmodifiableList(list)` guarantee?

- A) Deep copy — original unchanged
- B) Thread-safe access
- C) `UnsupportedOperationException` on mutation via wrapper
- D) Sorted order

Answer: C — Returns a view; mutations via original reference ARE visible. Only mutations via the wrapper are blocked.

M2. Best collection for word frequency in a multi-threaded app?

- A) `HashMap`
- B) `TreeMap`
- C) `ConcurrentHashMap`
- D) `LinkedHashMap`

Answer: C — `chm.merge(word,1,Integer::sum)` is atomic per key. `HashMap` requires external synchronization.

M3. What does `peek()` return on an empty `PriorityQueue`?

- A) Throws `NoSuchElementException`
- B) Throws `IllegalStateException`
- C) Returns null
- D) Returns 0

Answer: C — `peek()` and `poll()` return null on empty. `remove()` and `element()` throw `NoSuchElementException`.

M4. Virtual threads are best suited for?

- A) CPU-intensive number crunching
- B) IO-bound services with many blocking calls
- C) Replacing synchronized
- D) Parallel array sorting

Answer: B — Virtual threads park cheaply on blocking IO without consuming OS threads. For CPU-bound, platform threads + `ForkJoinPool`.

M5. Which statement about type erasure is correct?

- A) `List` and `List` are distinct at runtime
- B) `instanceof` works with parameterised types
- C) Generic info is erased to bounds at runtime
- D) Arrays are subject to erasure

Answer: C — Generics are compile-time only. At runtime both lists are just `List`. Arrays are reifiable — `String[]` is a distinct runtime type.

M6. Which GC for sub-millisecond pause requirement?

- A) Parallel GC
- B) G1GC
- C) Serial GC
- D) ZGC

Answer: D — ZGC (Java 15+): concurrent compaction, colored pointers — achieves sub-millisecond pauses. G1GC targets 100-200ms.

M7. HashMap treeification occurs when?

- A) Load factor exceeds 0.75
- B) Total size > 1000
- C) Bin size >= 8 AND table.length >= 64
- D) hashCode() returns same value for all keys

Answer: C — Both conditions required: TREEIFY_THRESHOLD=8 AND MIN_TREEIFY_CAPACITY=64. If table is small, resize happens instead.

M8. Correct way to increment a shared counter in multi-threaded code?

- A) volatile int count; count++
- B) int count; synchronized(this){count++}
- C) AtomicInteger count; count.incrementAndGet()
- D) Both B and C

Answer: D — Both B (synchronized) and C (AtomicInteger/CAS) are correct. AtomicInteger is generally faster due to lock-free CAS. volatile alone is NOT sufficient for count++ (non-atomic).

Quick Reference Cheat Sheet

Topic	Key Points to Remember
String	Immutable; pool via intern(); use equals(); StringBuilder for loops ($O(n)$ vs $O(n^2)$)
equals/hashCode	Override both; Objects.hash(); equal → same hashCode; used by all hash-based collections
ArrayList	$O(1)$ get; amortized $O(1)$ add; $O(n)$ mid-insert; 1.5x grow; NOT thread-safe
HashMap	$O(1)$ avg; treeify at $\text{bin} \geq 8 + \text{table} \geq 64$; load=0.75; allows null key; NOT thread-safe
ConcurrentHashMap	CAS+per-bin lock; no null key/value; atomic compute/merge; high-throughput
TreeMap/Set	Red-black tree; $O(\log n)$; NavigableMap/Set (floor/ceiling/headMap/tailMap)
LinkedHashMap	accessOrder LRU cache: new LinkedHashMap<>(16,0.75f,true)+removeEldestEntry
Generics/PECS	Erased to bounds at runtime; extends=read, super=write; no new T(); no T[]
volatile	Visibility only; NOT atomic; happens-before guarantee; use AtomicInteger for increment
ReentrantLock	Always unlock() in finally; tryLock(); fair constructor; Condition for wait/signal
CompletableFuture	Non-blocking; thenApply/Compose/Combine; exceptionally; allOf/anyOf; no .get()
Virtual Threads	Java 21; ~1KB; park cheaply on IO; newVirtualThreadPerTaskExecutor; IO-bound only
G1GC	Default Java 9+; regions; concurrent mark; -XX:MaxGCPauseMillis=200
ZGC	Sub-ms pauses; concurrent compact; Java 15+; -XX:+UseZGC; good for large heaps
Memory Leak	ThreadLocal.remove(); close resources; bounded caches; WeakReference; heap dump+MAT

End of Chapter 1 — Core Java & JVM Internals

Next: Chapter 2 — Design Patterns & Design Principles